

Offline REinforcement Optimization: OREO[4]

OREO is an offline reinforcement learning algorithm which leverages reasoning datasets with annotated rewards to improve the reasoning capabilities of Large Language Models.

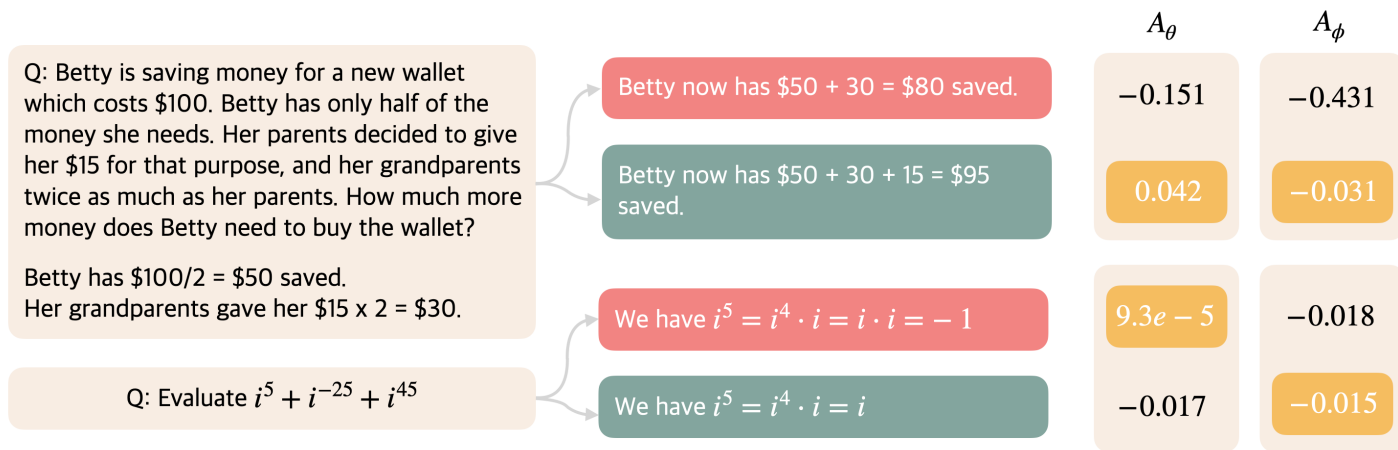


Figure 1. Advantage predicted by policy network A_θ and value network A_ϕ for reasoning traces

OREO trains a policy and a value network to produced the better reasoning. Through the Value network A_ϕ , OREO learns from both successfully and unsuccessful reasoning attempts. This differs from DPO which needs, for each input, a pair of positive and negative responses, which is cumbersome to generate in the case of reasoning. This also differs from rejection sampling which only keeps reasoning traces which were successful, and discards those that were unsuccessful. That is why OREO is the more efficient as it can use all data with minimal constraint compared to SFT, DPO or Rejection Sampling.

Value and Policy objective functions

MSE Loss is used as the objective for the Value Function along with a KL regularization term to stabilize training.

$$\mathcal{L}_V(\phi) = \frac{1}{T} \sum_{t=0}^{T-1} \left(V_\phi(s_t) - R_t + \beta \sum_{i \geq t} \log \frac{\pi_\theta(\mathbf{a}_i | \mathbf{s}_i)}{\pi_{\text{ref}}(\mathbf{a}_i | \mathbf{s}_i)} \right)^2$$

The policy objective:

$$\mathcal{L}_\pi(\theta) = \frac{1}{T} \sum_{t=0}^{T-1} \left(V_\phi(s_t) - R_t + \beta \log \frac{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\text{ref}}(\mathbf{a}_t | \mathbf{s}_t)} + sg \left[\beta \sum_{i > t} \log \frac{\pi_\theta(\mathbf{a}_i | \mathbf{s}_i)}{\pi_{\text{ref}}(\mathbf{a}_i | \mathbf{s}_i)} \right] \right)^2 + \alpha \mathcal{L}_{\text{reg}}$$

where $\mathcal{L}_{\text{reg}} = \frac{1}{T} \sum_{t=0}^{T-1} \text{KL}[\pi_\theta(\cdot | \mathbf{s}_t) \| \pi_{\text{ref}}(\cdot | \mathbf{s}_t)]$ is also a KL penalization term to stabilize training. Since the rewards are sparse, the policy objective uses Value function's predicted value as a reward signal.

Experiments

- **Supervised Fine Tuning** We first evaluate the performance of standard methods like supervised fine-tuning to serve as an additional baseline. SFT consists in using the Cross-Entropy Loss on successful reasoning traces. To make the model basic autoregressive output involve reasoning traces more often and of better quality
- **OREO training** Training and evaluation of the OREO method on the GSM8K and MATH datasets for Qwen2.5-1.5B-Instruct. Since OREO requires a policy and a value network, it is much more computationally expensive that SFT. That is why we use LoRA Finetuning with $r=128$ to fit into 24GB of VRAM, doing the entire training on a single consumer-grade graphics card.
- **test time compute evaluation** Using the value network, we do beam-search on reasoning traces to more efficiently explore the space of possible reasoning chains. Test-time compute is a new scaling paradigm popularized by models like o1 and o3 models of OpenAI, or QwQ from Qwen. However, their training recipe was not disclosed and OREO represents one of the first paper lifting the veil on how to leverage RL to achieve test-time compute scaling.

Base model and Datasets

As a base model, we Qwen2.5-Math-1.5B[5] which was used in the original release. To fit the training on a single graphics card we use the bfloat16 datatype and LoRA adapters. For training, a synthetic dataset of 150k examples is used to train the policy and value networks. Each data sample consists in a Question, an LLM response including chain-of-thought reasoning and the corresponding correctness reward (1 if the final answer was correct, 0 if incorrect).

For evaluation, we use the GSM8K[1] and MATH[3] datasets which consist of mathematical problems together with their corresponding solution.

GSM8K dataset: 7473 training problems and 1319 test problems, of elementary school difficulty.

MATH dataset: 7500 train and 5000 test problems of high-school difficulty.

Results

method	GSM8K	MATH
SFT	60.0	55.24
OREO	65.5	55.92

Table 1. accuracy on GSM8K and MATH for different training methods

We compare using supervised fine tuning and the OREO traning method on the GSM8K and MATH benchmarks. for a same number of trainset, OREO outperforms SFT on the GSM8K benchmark while performing similarly to SFT on the MATH benchmark. However, since OREO requires a critic network, it is significantly more computationally expensive using more than twice as much Memory and compute.

Test-time compute

We compare using supervised fine tuning and the OREO traning method on the GSM8K and MATH benchmarks. for a same number of trainset, OREO outperforms SFT on the GSM8K benchmark while performing similarly to SFT on the MATH benchmark. However, since OREO requires a critic network, it is significantly more computationally expensive using more than twice as much Memory and compute.

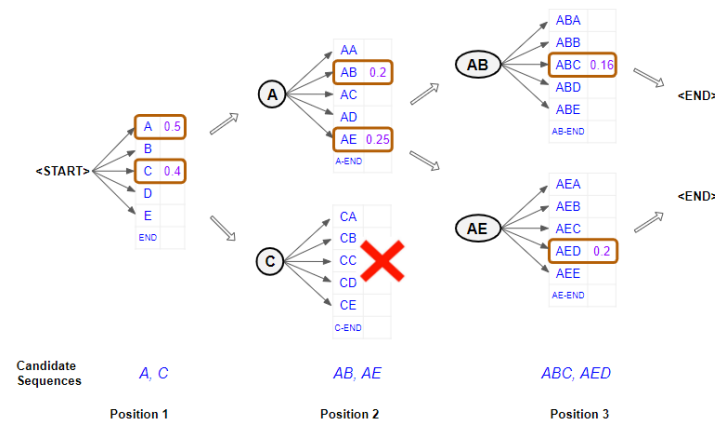


Figure 2. Visualization of beam search with a beam size of 2. [2]

At each reasoning step, B token trajectories are maintained. And for each trajectory, B next tokens are sampled. Then, out of the B^2 trajectories, the B with the highest value according to the value function are kept.

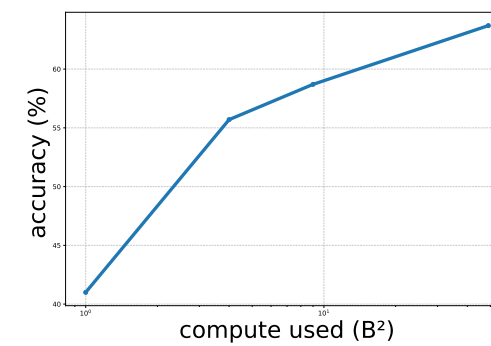


Figure 3. Accuracy over compute budget for GSM8K benchmark

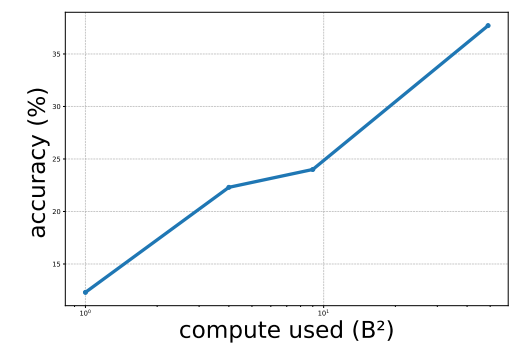


Figure 4. Accuracy over compute budget for MATH benchmark

Figure ?? shows that OREO successfully leverages the value function to reacher higher and higher accuracies as the beam size increases. This shows that the value function successfully distinguishes correct and incorrect reasoning steps. This opens a new way to achieve better performance while not having to scale the number. However, the compute budget necessary is considerable as simply evaluating both models took around 8 hours of a RTX3090.

Conclusion

RL with verifiable rewards brings better performance and sample efficiency than other methods like Supervised Fine Tuning and Rejection Sampling. It also allows us to scale test-time compute to improve performance which provides a new paradigm to scale performance without increasing the training cost.

References

- [1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [2] Ketan Doshi. Foundations of nlp explained visually: Beam search, how it works. Towards Data Science, April 2021.
- [3] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.
- [4] Huaijie Wang, Shibo Hao, Hanze Dong, Shenao Zhang, Yilin Bao, Ziran Yang, and Yi Wu. Offline reinforcement learning for llm multi-step reasoning, 2024.
- [5] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyu Xia, Xingzhang Ren, Xuanzhong Dou, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuxiang Liu, Zeyu Cui, Zhenru Zhang, and Zhiyuan Zhou. Qwen2.5-72b-instruct, 2025.